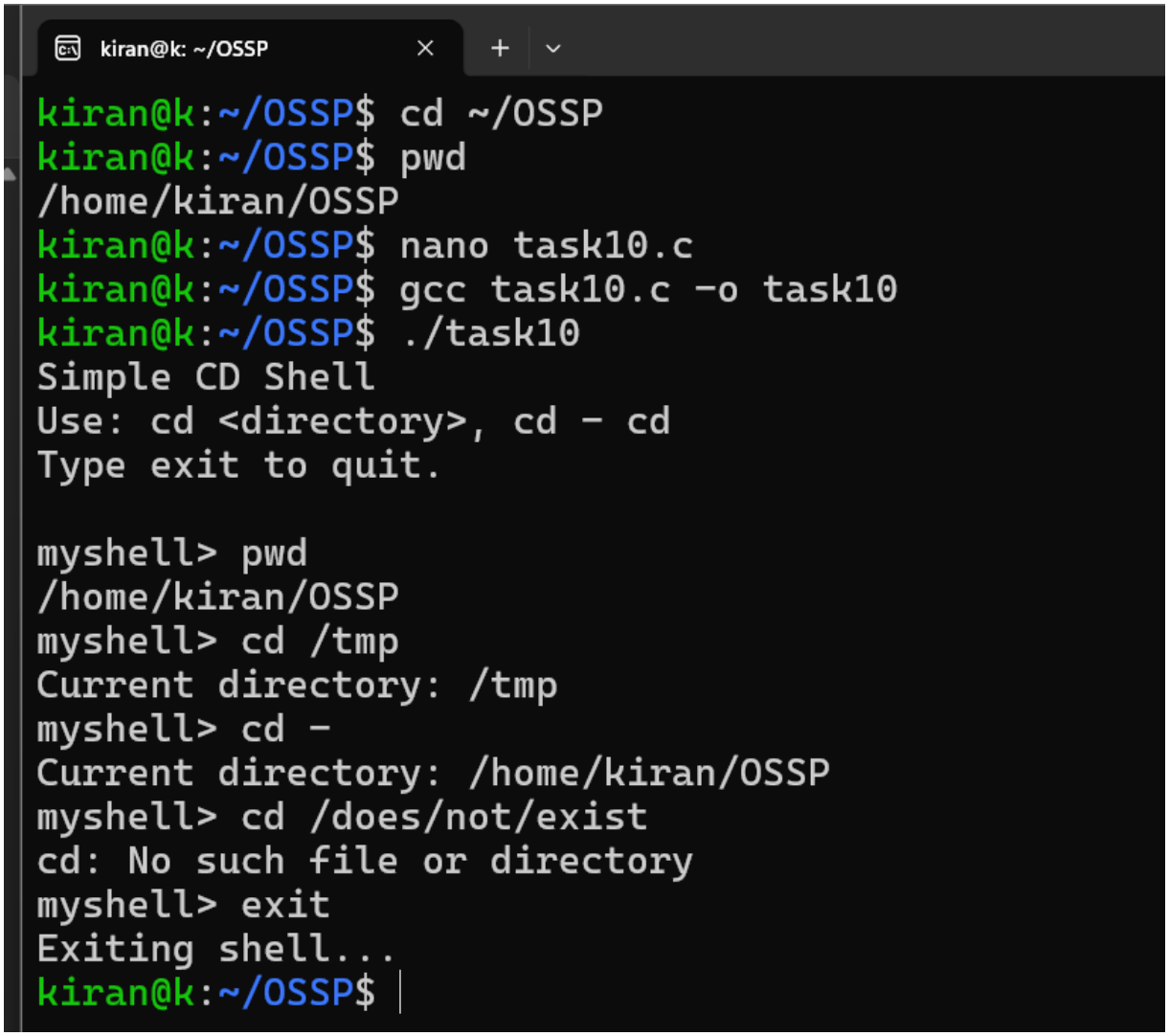
Skill Week-5

Session-9

Task 1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation



kiran@k: ~/OSSP

GNU nano 8.7.1 task10.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int change_directory(char *path) {
    char old_directory[PATH_MAX];
    char new_directory[PATH_MAX];

    /* Get current directory */
    if (getcwd(old_directory, sizeof(old_directory)) == NULL) {
        perror("getcwd");
        return 1;
    }

    /* If no path is given, go to HOME */
    if (path == NULL) {
        path = getenv("HOME");

        if (path == NULL) {
            printf("HOME variable is not set.\n");
            return 1;
        }
    }

    /* Handle cd - */
    if (strcmp(path, "-") == 0) {
        path = getenv("OLDPWD");

        if (path == NULL) {
            printf("OLDPWD is not set.\n");
            return 1;
        }
    }

    /* Change directory */
    if (chdir(path) != 0) {
        perror("cd");
        return 1;
    }

    if (getcwd(new_directory, sizeof(new_directory)) == NULL) {
        perror("getcwd");
        return 1;
    }
```

kiran@k: ~/OSSP

GNU nano 8.7.1 task10.c *

```c
    printf("Current directory: %s\n", new_directory);

    return 0;
}

int main() {
    char input[1024];
    char *command;
    char *path;

    printf("Simple CD Shell\n");
    printf("Use: cd <directory>, cd - cd\n");
    printf("Type exit to quit.\n\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        command = strtok(input, " \t");
        path = strtok(NULL, " \t");

        if (command == NULL)
            continue;

        if (strcmp(command, "cd") == 0) {
            change_directory(path);
        }
        else if (strcmp(command, "pwd") == 0) {
            char cwd[PATH_MAX];

            if (getcwd(cwd, sizeof(cwd)) != NULL)
                printf("%s\n", cwd);
            else
                perror("pwd");
        }
        else if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }
        else {
            printf("Invalid command: %s\n", command);
        }
    }

    return 0;
}
```

Task 2: TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
kiran@k:~/OSSP$ cp task11.c builtin_dispatch.c
kiran@k:~/OSSP$ gcc builtin_dispatch.c -o builtin_dispatch
kiran@k:~/OSSP$ ./builtin_dispatch
Built-in Command Shell
Available commands: cd, pwd, echo, export, exit

myshell> pwd
/home/kiran/OSSP
myshell> echo

myshell> cd /tmp
myshell> pwd
/tmp
myshell> echo

myshell> hello
Invalid command: hello
myshell>
```

kiran@k: ~/OSSP

```
  GNU nano 8.7.1                                task11.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int builtin_pwd(char **args)
{
    char cwd[1024];

    if (getcwd(cwd, sizeof(cwd)) != NULL) {
        printf("%s\n", cwd);
        return 0;
    }

    perror("pwd");
    return 1;
}

int builtin_cd(char **args)
{
    char *path;

    if (args[1] == NULL) {
        path = getenv("HOME");

        if (path == NULL) {
            printf("HOME is not set.\n");
            return 1;
        }
    }
    else if (strcmp(args[1], "-") == 0) {
        path = getenv("OLDPWD");

        if (path == NULL) {
            printf("OLDPWD is not set.\n");
            return 1;
        }
    }
```

kiran@k: ~/OSSP

GNU nano 8.7.1 task11.c *

```
    }
    else {
        path = args[1];
    }

    char oldpwd[1024];

    if (getcwd(oldpwd, sizeof(oldpwd)) == NULL) {
        perror("getcwd");
    }

    if (chdir(path) != 0) {
        perror("cd");
        return 1;
    }

    char newpwd[1024];

    if (getcwd(newpwd, sizeof(newpwd)) == NULL) {
        perror("getcwd");
        return 1;
    }

    setenv("OLDPWD", oldpwd, 1);
    setenv("PWD", newpwd, 1);

    return 0;
}

int builtin_echo(char **args)
{
    int i = 1;

    while (args[i] != NULL) {
        printf("%s", args[i]);

        if (args[i + 1] != NULL)
            printf(" ");

        i++;
    }
```

kiran@k: ~/OSSP

GNU nano 8.7.1 task11.c *

```
    printf("\n");
    return 0;
}

int builtin_export(char **args)
{
    if (args[1] == NULL) {
        printf("Usage: export NAME=value\n");
        return 1;
    }

    char *equal = strchr(args[1], '=');

    if (equal == NULL) {
        printf("Invalid export syntax.\n");
        return 1;
    }

    *equal = '\0';

    char *name = args[1];
    char *value = equal + 1;

    if (setenv(name, value, 1) != 0) {
        perror("export");
        return 1;
    }

    printf("Exported %s=%s\n", name, value);

    return 0;
}

int builtin_exit(char **args)
{
    printf("Exiting shell...\n");
    exit(0);
}

/* Dispatch table */
```

kiran@k: ~/OSSP

GNU nano 8.7.1 task11.c *

```c
struct builtin {
    char *name;
    int (*function)(char **);
};

struct builtin builtin_table[] = {
    {"cd", builtin_cd},
    {"pwd", builtin_pwd},
    {"echo", builtin_echo},
    {"export", builtin_export},
    {"exit", builtin_exit}
};

int builtin_count =
    sizeof(builtin_table) / sizeof(builtin_table[0]);

int execute_builtin(char **args)
{
    if (args[0] == NULL)
        return 1;

    for (int i = 0; i < builtin_count; i++) {
        if (strcmp(args[0], builtin_table[i].name) == 0) {
            return builtin_table[i].function(args);
        }
    }

    return -1;
}

/* Parse input */

void parse_input(char *input, char **args)
{
    int i = 0;

    args[i] = strtok(input, " \t\n");

    while (args[i] != NULL && i < MAX_ARGS - 1) {
        i++;
        args[i] = strtok(NULL, " \t\n");
```

```c
        args[i] = strtok(NULL, " \t\n");
    }

    args[i] = NULL;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Built-in Command Shell\n");
    printf("Available commands: cd, pwd, echo, export, exit\n\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (input[0] == '\n')
            continue;

        parse_input(input, args);

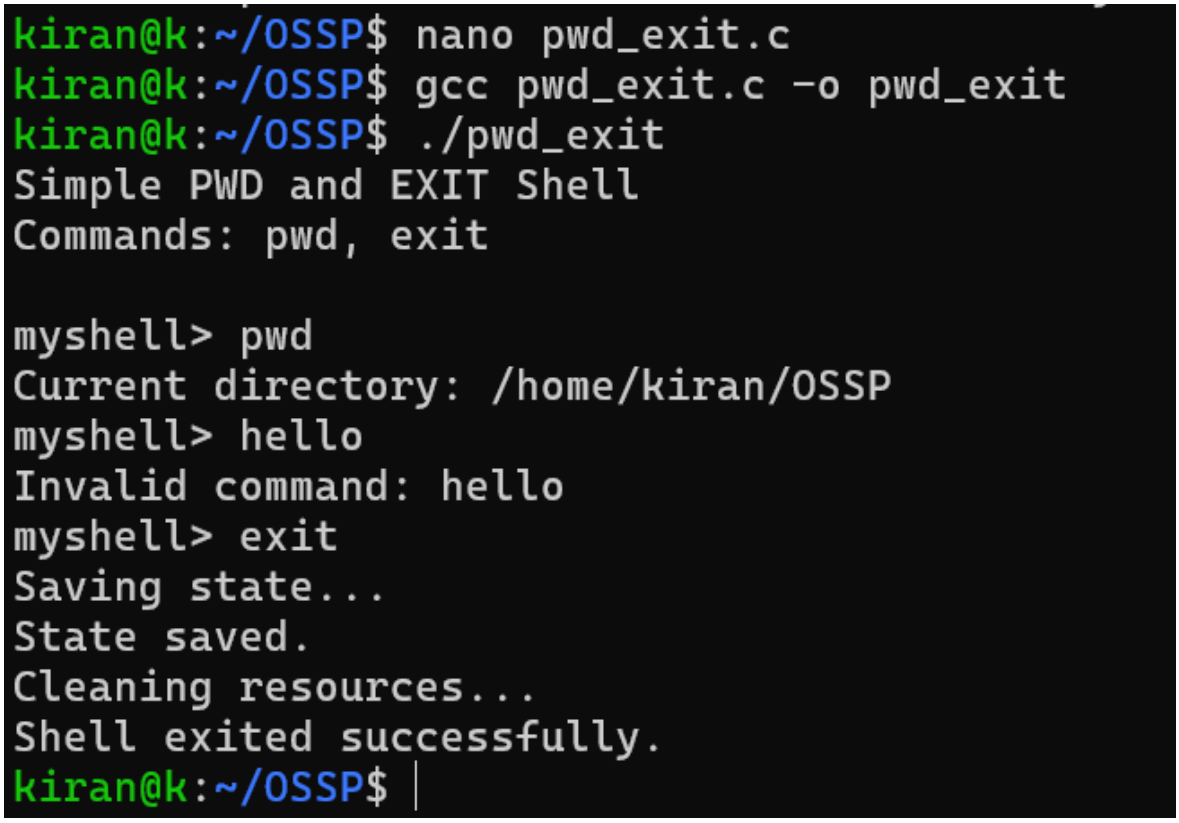
        int result = execute_builtin(args);

        if (result == -1)
            printf("Invalid command: %s\n", args[0]);
    }

    return 0;
}
```

Session-10

Task 1: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main() {

    char *buffer = NULL;
    size_t size = 0;

    printf("Simple PWD and EXIT Shell\n");
    printf("Commands: pwd, exit\n\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        ssize_t length = getline(&buffer, &size, stdin);

        if (length == -1) {
            printf("\nEnd of input. Exiting...\n");
            break;
        }

        buffer[strcspn(buffer, "\n")] = '\0';

        if (strcmp(buffer, "pwd") == 0) {

            char cwd[PATH_MAX];

            if (getcwd(cwd, sizeof(cwd)) != NULL) {
                printf("Current directory: %s\n", cwd);

                /* Save current directory */
                FILE *file = fopen("shell_state.txt", "w");

                if (file != NULL) {
                    fprintf(file, "%s\n", cwd);
                    fclose(file);
                }
            }
            else {
                perror("getcwd");
            }
```

```
                perror("getcwd");
            }
        }

        else if (strcmp(buffer, "exit") == 0) {

            printf("Saving state...\n");

            char cwd[PATH_MAX];

            if (getcwd(cwd, sizeof(cwd)) != NULL) {

                FILE *file = fopen("shell_state.txt", "w");

                if (file != NULL) {
                    fprintf(file, "%s\n", cwd);
                    fclose(file);
                    printf("State saved.\n");
                }
            }

            printf("Cleaning resources...\n");

            free(buffer);

            printf("Shell exited successfully.\n");

            return 0;
        }

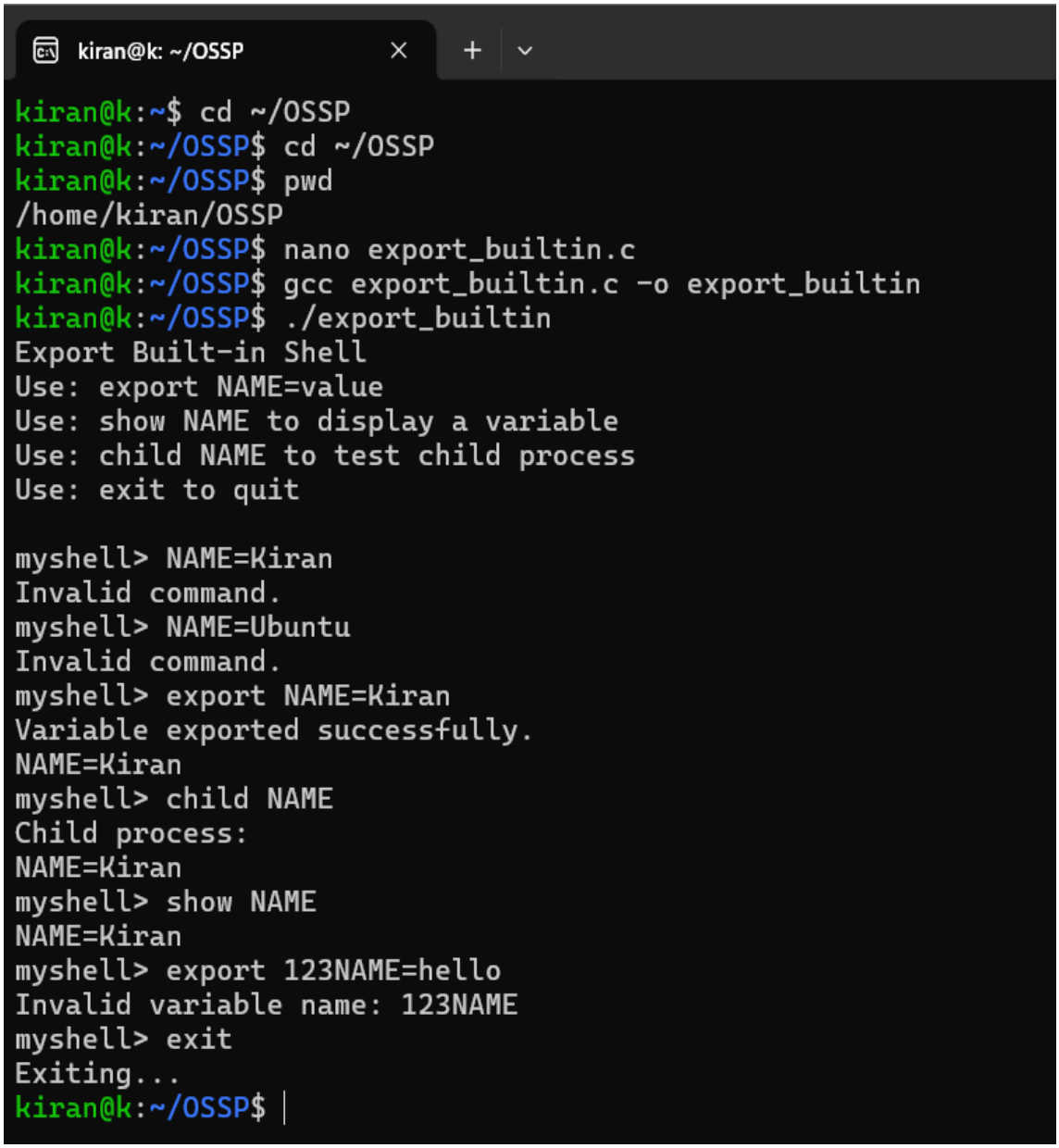
        else if (strlen(buffer) == 0) {
            continue;
        }

        else {
            printf("Invalid command: %s\n", buffer);
        }
    }

    free(buffer);

    return 0;
}
```

Task 2: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

int valid_name(const char *name) {

    if (name == NULL || name[0] == '\0')
        return 0;

    /* First character must be letter or underscore */
    if (!(isalpha(name[0]) || name[0] == '_'))
        return 0;

    /* Remaining characters */
    for (int i = 1; name[i] != '\0'; i++) {

        if (!(isalnum(name[i]) || name[i] == '_'))
            return 0;
    }

    return 1;
}

int builtin_export(char *argument) {

    if (argument == NULL) {
        printf("Usage: export NAME=value\n");
        return 1;
    }

    char *equal = strchr(argument, '=');

    if (equal == NULL) {
        printf("Invalid syntax. Use NAME=value\n");
        return 1;
    }

    /* Separate name and value */
    *equal = '\0';

    char *name = argument;
    char *value = equal + 1;
```

```
  GNU nano 8.7.1
    if (!valid_name(name)) {
        printf("Invalid variable name: %s\n", name);
        return 1;
    }

    /* Set environment variable */
    if (setenv(name, value, 1) != 0) {
        perror("setenv");
        return 1;
    }

    printf("Variable exported successfully.\n");
    printf("%s=%s\n", name, value);

    return 0;
}

void test_child_process(char *variable) {

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {

        char *value = getenv(variable);

        printf("Child process:\n");

        if (value != NULL)
            printf("%s=%s\n", variable, value);
        else
            printf("Variable not found.\n");

        exit(0);
    }

    waitpid(pid, NULL, 0);
}

int main() {

    char input[1024];
```

```
    printf("Export Built-in Shell\n");
    printf("Use: export NAME=value\n");
    printf("Use: show NAME to display a variable\n");
    printf("Use: child NAME to test child process\n");
    printf("Use: exit to quit\n\n");

    while (1) {

        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0) {
            printf("Exiting...\n");
            break;
        }

        if (strncmp(input, "export ", 7) == 0) {

            char argument[1024];

            strcpy(argument, input + 7);

            builtin_export(argument);
        }

        else if (strncmp(input, "show ", 5) == 0) {

            char *name = input + 5;
            char *value = getenv(name);

            if (value != NULL)
                printf("%s=%s\n", name, value);
            else
                printf("%s is not defined.\n", name);
        }
```

```
        else if (strncmp(input, "child ", 6) == 0) {

            char *name = input + 6;

            test_child_process(name);
        }

        else if (strlen(input) == 0) {
            continue;
        }

        else {
            printf("Invalid command.\n");
        }
    }

    return 0;
}
```